

Working with LLMs

Understand, translate, and validate SAS workflows

For data science professionals who know SAS
and are new to LLM collaboration.

**Teach a collaboration workflow,
not a collection of magic prompts.**

Concept and sample learning aids for team review. Not an approved rollout.

4

brief sessions

Python first

Model-neutral
Evidence-led

Reduce open choices. Make checks reproducible.

**We are reducing unnecessary choices and building reproducible checks.
We are not promising identical model responses.**

01

Understand

Review what the SAS program does.

02

Specify

State the behavior and acceptance rules.

03

Translate

Implement one agreed unit in Python.

04

Check

Run reviewed code against expectations.

05

Improve

Refactor only from a checked baseline.

A different implementation can pass. A repeated implementation can still be wrong.

The acceptance decision belongs to the analyst and is limited to the stated scope.

Four brief sessions, four visible outcomes

Approximately 25 minutes each. Access and basic environment setup happen before class.

SESSION	WHAT THE LEARNER PRODUCES
01 Read before you rewrite	A reviewed explanation and an unresolved-dependency list.
02 Ask for the behavior you need	A bounded MEAN translation request and candidate code.
03 Check more than “it runs”	An evidence record using SUBSTR or LENGTH checks.
04 Work through a small pipeline	A decomposed workflow with boundary and final-output checks.

5 min explain + 8 min demonstrate + 8 min practice + 4 min discuss

Make the consequential choices inspectable

MEAN is the first worked example. The guide contains the complete reusable request.

THE FAMILIAR SAS CALCULATION

```
average = mean(x, y, z);
```

1	2	3	Three rows.
4	.	6	One calculation.
.	.	.	Different missingness.

PRACTICAL HEADINGS, NOT AN ACRONYM

Task	One transformation.
Source and context	Source, schema, dependencies.
Environment	Approved Python and libraries.
Required behavior	Missingness, order, types, outputs.
Deliverable	Code and a requirement mapping.
Uncertainty and checks	Flag gaps; label evidence honestly.

HOW

Name the behavior to preserve.

WHY

The choice becomes reviewable.

WHY NOT

A fluent answer is not a check.

Check the result, not the prose

Use fixed expectations and a reviewed implementation. Keep ordinary, edge, and preservation checks distinct.

INPUT x, y, z	EXPECTED
1, 2, 3	2.0
4, missing, 6	5.0
all missing	missing
0, missing, missing	0.0
missing, -6, missing	-6.0

T1 / KNOWN ANSWERS
Original values and missing positions.

T2 / DISCRIMINATING CASES
Zero and a lone negative value.

T3 / PRESERVATION
Input unchanged; output structure retained.

Three named tests do not prove universal equivalence.

Python reference: locally checked | SAS baseline: pending | Live model trial: pending

A compact packet, with an executable companion

One set of requirements and examples, presented in formats people can use.

4-PAGE FIELD GUIDE

The workflow, practical requests, checks, and repairs.

2-SIDED DESK REFERENCE

Prompt headings on the front; evidence and repair on the back.

EXISTING WORKED MODULES

Use selected examples and appendices as needed.

MEAN NOTEBOOK PROTOTYPE

01	SAS source and required behavior
02	Prompt and saved-response record
03	Reviewed Python candidate
04	Executed checks and observed results
05	Evidence, limits, and acceptance decision

Static HTML preview included. No model API connection is required for the supplied demonstration.

Keep the pilot small enough to teach

Add languages and infrastructure only when they serve a defined learning objective.

CORE FOR REVIEW

Python as the first target.

Model-neutral collaboration.

Synthetic, bounded examples.

Evidence-led acceptance.

OPTIONAL DELIVERY

Jupyter as a companion.

Portal plus reviewed execution.

VS Code and SSH access.

Setup handled before class.

LATER EXTENSIONS

DuckDB for analytical SQL.

R, Stata, and Julia tracks.

Macros and stateful DATA steps.

Performance and redesign.

Analyst: requirements and acceptance. Model: proposed work. Runtime: observed execution.

Decide the pilot, not an entire AI curriculum

Use these materials to review scope, pacing, prerequisites, and evidence expectations.

AUDIENCE AND PACE	Are four short sessions realistic for these learners? Which existing examples should anchor the pilot?
ENVIRONMENT AND SUPPORT	What approved portal and execution path will be used? Who will confirm access and resolve semantic questions?
ACCEPTANCE AND FEEDBACK	What must learners demonstrate before completion? What observations will drive changes after the pilot?

The learner can direct the work, inspect the evidence,
and recognize when the translation is not ready to trust.

01 / UNDERSTAND THE ASSIGNMENT

What am I asking the model to do?

You know the analysis. Make the required behavior explicit, then inspect the evidence.

THE WORKING PROMISE

We are reducing unnecessary choices and building reproducible checks. We are not promising identical model responses.

Teach a collaboration workflow, not a collection of magic prompts. Ask for one kind of work at a time, and review the result before moving to the next stage.

Stage	Ask for	Review before moving on
Understand	Explain inputs, transformations, outputs, and dependencies.	Does the explanation match the source? What is inferred?
Specify	State the behavior, environment, scope, and expected results.	Which choices must the analyst decide?
Translate	Implement one agreed unit in Python.	Does the code address the requirements without unrelated changes?
Check	Execute reviewed code against established expectations.	What passed, what failed, and what remains unchecked?
Improve	Propose a separate refactor or optimization.	Do required behaviors still hold after the change?

Start with explanation, not a rewrite

Explain this SAS step without translating it. Identify its inputs, output columns, transformations, and dependencies. Separate statements supported by the code from assumptions. Flag missing definitions before describing affected behavior.

How: Check the explanation against the source and relevant documentation. Treat comments as context to verify, not automatic proof of behavior.

Why: An implementation can look reasonable while solving a different problem. Agreeing on the task makes that mismatch easier to notice.

Why not: Do not ask for translation, a new architecture, and a different analytical method in one request. You would lose a clear basis for comparison.

KEEP THE UNIT COHERENT

Chunk at an understandable input/output boundary, not an arbitrary line count. Carry needed formats, macro definitions, grouping and ordering assumptions with the chunk. Ask for missing context rather than inventing it.

Use only approved source and synthetic or otherwise authorized data. A model response is a proposal, not permission to run code or access additional data.

02 / SPECIFY THE REQUEST

What does the model need to know?

Use practical headings. Make consequential choices explicit, not every possible choice.

Completed example: row-wise MEAN

Heading	A bounded request you can adapt
Task	Translate only the row-wise SAS MEAN calculation into a Python function named <code>candidate(df)</code> .
Source and context	Use the attached SAS source. The Python input contains <code>case_id</code> , <code>row_id</code> , <code>x</code> , <code>y</code> , <code>z</code> . Only <code>x</code> , <code>y</code> , <code>z</code> are measurements; the other columns are labels.
Environment	Use Python with pandas in the approved environment. The measurement columns are float64 and use NaN for ordinary numeric missing values. Record actual versions separately.
Required behavior	Average <code>x</code> , <code>y</code> , <code>z</code> within each row, excluding missing values. Keep an all-missing average missing. Preserve zeros and negatives. Return a new table without changing the input; retain its rows, order, index and columns, and append numeric average.
Deliverable	Return code and a brief mapping to the requirements. Do not install packages, read unrelated files, create files, or execute code in this step.
Uncertainty and checks	Flag missing information or conflicts before implementing affected behavior. Do not silently invent a rule. State which checks are proposed, not executed. Use the supplied expected values; do not alter them to match the implementation.

The source to attach or paste after the request

```
data test;
  input x y z;
  average = mean(x, y, z);
  datalines;
1 2 3
4 . 6
. . .
;
run;
```

Expected averages: 2.0, 5.0, and missing. The notebook adds input labels and preservation requirements for testing; these are explicit interface requirements, not hidden SAS behavior. [SAS; Project]

How: Supply the actual source, schema, behavior rules, and approved tools. Include only the context needed for the bounded task.

Why: Stated choices can be inspected. Unstated choices must be found before they can be accepted.

Why not: More text is not automatically more useful. Remove conflicting instructions and unrelated conversation history. A concise prompt that passes the checks is a valid outcome.

Sources: [SAS](#) | [Project](#)

03 / CHECK THE EVIDENCE

How do I check the result?

Define expected behavior before judging the candidate. Keep the expected values separate from its calculation.

THREE DIFFERENT CLAIMS

The model displayed an answer. The code executed. The observed results met specified checks. Record only the claim supported by the evidence you actually have.

MEAN: make the checks visible

Input x, y, z	Expected average	What this distinguishes
1, 2, 3	2.0	Ordinary arithmetic.
4, missing, 6	5.0	Exclude missing values; do not use a fixed denominator of three.
missing, missing, missing	Missing	Do not turn an all-missing row into zero.
0, missing, missing	0.0	Zero is an observed value.
missing, -6, missing	-6.0	Preserve a lone negative measurement.

Expected values above follow the stated MEAN rule and the shared repository fixture. They are not captured SAS output. [SAS; Fixture]

Three named acceptance tests

T1. Known answers: Check all three original rows, including their missing positions.

T2. Discriminating cases: Check the added zero and negative cases.

T3. Preservation: Check that the input is unchanged and that a new output preserves labels, values, row order, index and types while adding average. [Project]

For this fixture, pandas computes across columns with axis=1 and excludes missing values with skipna=True. Check missing results with pandas.isna(), not equality to NaN. [pandas; Missing]

How: Inspect the code, execute it in the approved environment, and retain the actual output. Review implementation requirements as well as values. For other calculations, agree numerical tolerances before comparing.

Why: A check is useful when it can reject a relevant mistake. Include cases that distinguish plausible interpretations, not only easy inputs.

Why not: Do not let the candidate implementation generate its own expected answers. Model-suggested tests need independent review; a second model is not an execution record.

ACCEPTANCE HAS A BOUNDARY

Three tests are a teaching structure, not proof of general equivalence. Record the supported scope and unresolved limits. A teaching pass is not production approval.

Sources: [SAS](#) | [Fixture](#) | [Project](#) | [pandas](#) | [Missing](#)

04 / REVISE OR PAUSE

What do I do when something is wrong?

Use a mismatch as evidence. Request the smallest justified correction.

A BOUNDED REPAIR REQUEST

For input [4, missing, 6], the required average is 5.0, but this run returned 3.3333333333. The attached code filled missing values with zero. Explain the mismatch, then revise only the calculation. Preserve the interface and existing checks. Return code without executing it. Do not change expected values.

How: Preserve the original response and failed output. Include the source/input version, expected result, actual result and relevant code or traceback. Rerun every acceptance test after correction.

Why: A narrow change is easier to review than a replacement program. Rerunning the earlier checks helps reveal regressions.

Why not: Do not keep saying “try again” without evidence, or change expected values to make a failure disappear. A genuine correction to a requirement must be reviewed and versioned separately.

Pause when a consequential choice is unresolved

Missing macro or format definitions; conflicting source behavior and intended behavior; unknown join or ordering rules; an unapproved dependency; or changes to the analytical method. Identify the question and its owner. Do not guess to keep the conversion moving.

Keep the conversation, workspace, and runtime distinct

For controlled A/B trials, keep the source, starting instructions, model, and visible settings fixed. Start fresh conversations, avoid carrying prior outputs forward, and isolate workspaces when tools can read files. This does not guarantee identical responses. Bedrock parameters differ by model; record unavailable settings as unknown. [AWS]

In a notebook, inspect the cells, restart the kernel, then run from the beginning. Kernel restart clears in-memory state, not files on disk. Review saved outputs before sharing: notebook documents can retain them. [Jupyter]

Close each exercise with an evidence record

Required behavior	Evidence	Limits	Decision
What had to remain true?	What ran, in which environment, and what happened?	What was not checked or remains unresolved?	Accept for the stated teaching scope, revise, or seek clarification.

Keep with the record: model identifier, access application, visible settings, source/prompt/code versions, run output, and reviewer decision. Do not paste credentials into the record.

Sources: [AWS](#) | [Jupyter](#)

FRONT / DIRECT THE WORK

A practical prompting reference

Understand > Specify > Translate > Check > Improve

START WITH ONE KIND OF WORK

Explain first when the behavior is unclear. Translate only after agreeing on requirements. Request optimization separately from a checked baseline.

Copy these headings; fill only what matters

Heading	What to provide
Task	The specific work and its boundary: explain, decompose, translate, diagnose, or improve.
Source and context	Actual SAS source; relevant definitions; input schema; upstream dependencies; downstream expectations. Use only approved inputs.
Environment	Approved language and libraries; actual versions; available execution tools. State restrictions rather than letting the model install its preferences.
Required behavior	Missing values; ordering; grouping; types; calculations; output structure. Name any intended behavior change instead of disguising it as translation.
Deliverable	The exact output: explanation, code, mapping, or diagnosis. Specify whether this step is response-only and exclude unrelated changes.
Uncertainty and checks	Flag missing context and conflicts. State expected results and evidence requirements. Do not invent a rule or claim checks were run without an execution record.

Three useful starters

Understand: “Explain this SAS step without rewriting it. Identify inputs, outputs and dependencies. Separate source-supported behavior from assumptions.”

Translate: “Implement only the agreed transformation in Python. Preserve the stated behavior and interface. Return code without executing it.”

Diagnose: “For this input, expected is X and observed is Y. Use the attached code and output to identify the mismatch. Change only the affected behavior.”

How: State consequential choices in ordinary language.

Why: Explicit choices are easier to inspect and check.

Why not: Long prompts, role-play, and confident wording are not substitutes for requirements and evidence.

BEFORE YOU PRESS SEND

Is the input authorized? Is the task bounded? Is relevant context present? Have you excluded credentials and unrelated files? Are the expected outputs or acceptance rules clear?

Print at actual size on US Letter. Duplex: flip on the long edge. This sheet is a pilot aid, not an approved operating procedure.

[BACK / CHECK THE RESULT](#)

An evidence and repair reference

Different code may meet the same requirements. Repeated code may still be wrong.

Check	Ask yourself	MEAN example
Known answers	Did all expected fixture values match, including missing positions?	1, 2, 3 -> 2.0 4, missing, 6 -> 5.0 All missing -> missing
Discriminating cases	Would the checks catch a plausible but wrong interpretation?	Zero remains zero. A lone -6 remains -6.
Preservation	Did required inputs and structure survive?	No input mutation. Same labels, row order and index. Numeric average appended.

These are the MEAN pilot requirements and shared fixture expectations, not a record of SAS execution. For NaN, use `pandas.isna()` instead of equality. [SAS; Fixture; Missing]

Repair with evidence

Preserve the failed result. Provide the relevant source, inputs, expected and observed outputs, code, and error details. Ask for a bounded diagnosis and correction. Inspect the change, then rerun all checks.

Pause rather than guess

Seek clarification for missing definitions, conflicting requirements, unclear ordering or joins, unapproved dependencies, or changes to the analytical method. A reviewed change in expectations needs a new version.

Record the state, not just the answer

Keep model identifier, access application, visible settings, source/prompt/code versions, execution output and review decision. Mark unknown settings as unknown; Bedrock settings vary by model. [AWS]

Fresh conversation does not mean clean workspace. For notebooks, inspect cells, restart the kernel and run top to bottom. Restart does not delete files. Review stored outputs before sharing. [Jupyter]

Close the exercise

Record	Write one useful sentence
Required behavior	What needed to remain true?
Evidence	What did I run and what did it show?
Limits	What remains untested or unresolved?
Decision	Accept for the stated teaching scope, revise, or seek clarification.

DO NOT TURN A FAILED CHECK INTO A MOVING TARGET

Keep expected results separate from the candidate. Do not weaken checks just to obtain a pass. Three named tests do not prove universal equivalence or authorize production use.

Sources: [SAS](#) | [Fixture](#) | [Missing](#) | [AWS](#) | [Jupyter](#)